

HPC Unit 2

HPC Unit 2 – Memory Hierarchy Design

Ten Advanced Cache Optimizations

$$AMAT = \text{Hit Time} + \text{Miss Rate} \times \text{Miss Penalty}$$

Each optimization targets one or more of: Hit Time (HT), Miss Rate (MR), Miss Penalty (MP).

Optimization 1: Small and Simple L1 Cache → Reduce Hit Time & Power

Why small and simple?

The critical timing path on a cache hit involves three steps:

1. **Index decoding** — select the correct set
2. **TAG comparison** — check if the block is a hit
3. **MUX selection** — choose the correct way (only in set-associative)

Reducing either cache size or associativity shortens all three steps.

Parameter	Effect on Hit Time	Effect on Power
Smaller cache	Fewer sets → faster index decode	Less energy per access
Lower associativity	Fewer TAGs to compare, no/fewer MUX stages	Fewer comparators active

Direct Mapped vs Set Associative: In DM cache, indexing and TAG comparison happen in parallel and data is kept ready for immediate delivery — lower HT but higher MR. In 4-way SA, four comparisons + MUX → higher HT, lower MR.

Key Tradeoff: Access time and energy both increase as cache size and associativity increase.

Example:

Given: 32 KB 4-way SA, 32 KB 2-way SA. Miss penalty to L2 = 15× access time of faster cache. MR: 2-way = 3.8%, 4-way = 3.7%. HT of 4-way is 1.4× HT of 2-way.

Let HT of 2-way = t :

$$AMAT_{2\text{-way}} = t + 0.038 \times 15t = t(1 + 0.57) = 1.57t$$

$$AMAT_{4\text{-way}} = 1.4t + 0.037 \times 15t \times 1.4 = 1.4t(1 + 0.555) = 2.177t$$

→ **2-way SA has faster AMAT** despite higher MR, because the HT difference dominates.

Optimization 2: Way Prediction → Reduce Hit Time

Idea: In a set-associative cache, instead of checking all ways in parallel every time, *predict* which way will be a hit based on past access history. Only check that way first — effectively behaving like a Direct Mapped cache.

Mechanism:

- Extra **prediction bits** stored per set
- On access, MUX is pre-set to predicted way; only **one TAG comparison** done (like DM)
- If prediction is **correct** → 1 clock cycle (DM-speed hit time)
- If prediction is **wrong** → pay 1 extra penalty cycle to check remaining ways

$$AMAT_{\text{way pred}} = (HT_{DM} \times P_{\text{correct}} + HT_{SA} \times P_{\text{wrong}}) + MR_{SA} \times MP$$

Example:

- 4-way SA: HT = 2 cycles, MR = 3.3%, MP = 10 cycles
- Way predictor: 80% accuracy, miss-predicted hit takes 1 extra cycle (HT = DM + 1)

$$AMAT_{4\text{-way}} = 2 + 0.033 \times 10 = 2.33 \text{ cycles}$$

$$AMAT_{\text{way pred}} = (1 \times 0.8 + 2 \times 0.2) + 0.033 \times 10 = 1.2 + 0.33 = 1.53 \text{ cycles}$$

→ Way prediction is faster! Power is also saved by turning off (power gating) the unpredicted ways.

Optimization 3: Pipelined Cache Access → Increase Cache Bandwidth

Idea: Break cache access into independent pipeline stages so multiple accesses can be in-flight simultaneously (like CPU pipelining).

Cache access stages:

1. Address Generation
2. TLB Read / Translation

3. Index + TAG Comparison (Hit/Miss decision)

4. Data Transfer

Advantage: Higher clock frequency → higher bandwidth

Disadvantage: Latency for a single hit increases (multiple cycles instead of 1) → worse branch miss penalty and load-use hazard penalty

Used heavily in modern high-frequency processors where bandwidth > latency is the priority.

Optimization 4: Non-Blocking Cache → Increase Cache Bandwidth

In a normal (blocking) cache, the processor **stalls completely** on a cache miss until the data returns from the next level.

Non-blocking (lockup-free) cache allows the processor to continue issuing requests even while a miss is being serviced.

Two levels of non-blocking:

Type	Description	Benefit
Hit Under Miss	While miss is in progress, subsequent cache hits are still serviced	Reduces effective miss penalty
Hit Under Multiple Miss	Multiple simultaneous misses are tracked and serviced concurrently	Further reduces effective miss penalty for out-of-order processors

Miss Status Holding Register (MSHR): Hardware structure in L1 that tracks outstanding misses. L2 must also support concurrent miss servicing for this to work.

Empirical result: Allowing one hit-under-miss reduces miss penalty by ~9% (integer) and ~12.5% (FP). Allowing a second miss improves to ~10% and ~16%. Beyond ~64 outstanding misses there's little further gain.

Optimization 5: Multi-Banked Cache → Increase Cache Bandwidth

Idea: Divide the cache into N independent banks, each with its own port. Multiple accesses can proceed in parallel if they target different banks.

Sequential Interleaving: Address block b maps to bank $b \bmod N$.

Example (4 banks):

- Bank 0 → block addresses where $\text{addr} \bmod 4 = 0$
- Bank 1 → block addresses where $\text{addr} \bmod 4 = 1$
- Bank 2 → block addresses where $\text{addr} \bmod 4 = 2$
- Bank 3 → block addresses where $\text{addr} \bmod 4 = 3$

Benefit: Spatial locality naturally spreads accesses across banks → near-parallel cache bandwidth.

Bonus: Multiple banks also reduce power since only one bank is activated per access (instead of the whole monolithic cache).

Note: Simultaneous access to the **same** bank still causes a conflict stall.

Optimization 6: Critical Word First & Early Restart → Reduce Miss Penalty

On a cache miss, the processor needs only **one specific word** in the fetched block, but the whole block must be transferred.

Strategy: Don't wait for the full block — send the needed word to the processor as soon as it arrives, and let the processor continue while the rest of the block is fetched in the background.

Two techniques:

Critical Word First:

1. Request the *missed word* (critical word) first from memory
2. Send it to the processor immediately when it arrives → processor resumes
3. Continue fetching remaining words in the background (overlapped with execution)

Advantage: Maximum overlap of memory transfer and CPU execution.

Drawback: Complex to implement (reordering, locating critical word, placing it correctly in the cache block, then filling remaining words). Also, if a second miss references a word still being fetched, the miss penalty is the non-overlapped time.

Early Restart:

1. Fetch words in normal order
2. As soon as the requested word arrives, send it to the processor and resume

Advantage: Simpler to implement than Critical Word First.

Drawback: Less overlap — if the requested word happens to be at the end of the block, most of the miss penalty is unavoidable.

Both techniques only give significant benefit when **block sizes are large**.

Optimization 7: Merging Write Buffer → Reduce Miss Penalty

Context: Write-through caches use a write buffer. Each store writes an address + data entry to the buffer, which drains to main memory in the background.

Problem without merging: Four sequential stores to addresses $M[100]$, $M[108]$, $M[116]$, $M[124]$ fill **four separate buffer entries**, even though all four addresses are contiguous and belong to the same block. The buffer is "full" even though only $\frac{1}{4}$ of each entry's capacity is used.

Write Merging: The write buffer controller checks if the new store's address is *contiguous* with an existing buffer entry. If so, the new data is **merged** into the existing entry rather than creating a new one.

Effect of merging: The four stores above merge into **one buffer entry** → three entries freed up → less stalling, better memory utilization.

Writes without merging: 4 entries used

Writes with merging: 1 entry used

Limitation: Cannot be applied to **I/O device registers** mapped into the physical address space, because individual I/O registers are not interchangeable memory locations — they may have side effects when written.

Optimization 8: Compiler Optimizations → Reduce Miss Rate

Software-based techniques that restructure code to improve cache behavior.

a) Loop Interchange

Problem: Row-major 2D arrays in memory but the inner loop iterates over rows (column-major access) → elements accessed are far apart in memory → spatial locality destroyed.

Example: `x[5000][100]` stored row-major. Original loop:

```
for (j = 0; j < 100; j++)
    for (i = 0; i < 5000; i++)
        x[i][j] = 2 * x[i][j];
```

Access pattern: $x[0][0]$, $x[1][0]$, $x[2][0]$, ... — each element 100 words apart (may not fit in a single cache block).

Fix: Swap the loop order:

```
for (i = 0; i < 5000; i++)
    for (j = 0; j < 100; j++)
        x[i][j] = 2 * x[i][j];
```

Now accesses $x[0][0]$, $x[0][1]$, $x[0][2]$, ... — sequential, exploiting spatial locality.

No change in instruction count — pure cache performance improvement.

b) Blocking (Tiling)

Problem: Matrix operations (e.g., matrix multiply) access rows of one matrix and columns of another simultaneously. Simple loop interchange doesn't fully solve this because both row and column accesses exist.

Idea: Divide matrices into small **sub-blocks (tiles)** that fit in the cache. Process one tile at a time to maximize reuse before eviction.

Original matrix multiply:

```
for (i = 0; i < N; i++)
    for (j = 0; j < N; j++) {
        r = 0;
        for (k = 0; k < N; k++)
            r += y[i][k] * z[k][j];
        x[i][j] = r;
    }
```

Worst case memory words accessed: $2N^3 + N^2$

Blocked version (block size B):

```
for (jj = 0; jj < N; jj += B)
    for (kk = 0; kk < N; kk += B)
        for (i = 0; i < N; i++)
            for (j = jj; j < jj+B; j++) {
                r = 0;
                for (k = kk; k < kk+B; k++)
```

```
        r += y[i][k] * z[k][j];  
    x[i][j] += r;  
}
```

Worst case memory words accessed: $\frac{2N^3}{B} + N^2 \rightarrow$ improvement by factor $\sim B$ (exploits temporal locality within each tile).

Optimization 9: Hardware Prefetching → Reduce Miss Penalty / Miss Rate

Idea: Fetch blocks *before* the processor requests them, hiding the miss penalty.

Instruction Prefetching:

- On an instruction cache miss, fetch **two blocks**: the requested block + the next consecutive block
- Requested block → goes into **instruction cache**
- Next block → goes into an **instruction stream buffer** (separate small buffer)
- If the next access hits in the stream buffer → cancel the cache request, use the buffer data, issue the next prefetch

Instruction prefetching is effective because instructions are accessed mostly sequentially.

Data Prefetching:

- Similar approach but harder to predict access patterns
- Compiler hints can improve prefetch accuracy (see Optimization 10)

Cautions:

- Prefetching that **interferes with demand misses** can actually hurt performance
 - Unused prefetched data wastes memory bandwidth and power
-

Optimization 10: Compiler-Controlled Prefetching → Reduce Miss Penalty / Miss Rate

Idea: The compiler inserts explicit `prefetch` instructions ahead of time so data is in cache when needed.

Two flavors:

- **Register prefetch:** Loads value into a register

- **Cache prefetch:** Loads data into cache only (not a register)

Requirements:

- Cache must be **non-blocking** (otherwise prefetch stalls the processor)
- Most effective for **loops** (predictable access patterns)

Strategy: Prefetch data d iterations ahead, where $d = \text{miss penalty} / \text{loop iteration time}$.

Example: Array $a[3][100]$, $b[101][1]$, block size = 16B, elements = 8B (2 elements per block).

Without prefetching:

- Array a : 150 misses (spatial locality $\rightarrow$ miss on even j , hit on odd j , 3 rows $\times$ 50)
- Array b : 101 misses
- **Total: 251 misses**

With prefetching (prefetch 7 iterations ahead):

```
// i = 0
for (j = 0; j < 100; j++) {
    prefetch(b[j+7][0]);
    prefetch(a[0][j+7]);
    a[0][j] = b[j][0] * b[j+1][0];
}
// i = 1, 2
for (i = 1; i < 3; i++)
    for (j = 0; j < 100; j++) {
        prefetch(a[i][j+7]);
        a[i][j] = b[j][0] * b[j+1][0];
    }
```

- 7 misses for $b[0][0] \dots b[6][0]$, 4 misses for $a[0][0] \dots a[0][6]$
- b not re-prefetched for $i=1,2$ (already in cache from $i=0$)
- **Total: 19 non-prefetched misses** (vs 251 original)

Speedup calculation:

- Original: 2100 execution cycles + 25100 miss penalty cycles = **27200 cycles**
- Prefetched: 900 + 1600 execution + 1100 + 800 miss penalty = **4400 cycles**

$$\text{Speedup} = \frac{27200}{4400} \approx 6.2\times$$

Cost: 400 extra prefetch instructions — well worth it.

Cache Optimization Summary

Optimization	Reduces	Complexity
Small & simple L1	Hit Time, Power	Low
Way prediction	Hit Time	Medium
Pipelined access	Hit Time (bandwidth)	Medium
Non-blocking cache	Miss Penalty	High
Multi-banked cache	Miss Penalty (bandwidth)	Medium
Critical word first	Miss Penalty	High
Merging write buffer	Miss Penalty	Low
Loop interchange	Miss Rate	Low (compiler)
Blocking	Miss Rate	Medium (compiler)
Hardware prefetching	Miss Penalty / Miss Rate	Medium
Compiler prefetching	Miss Penalty / Miss Rate	Medium (compiler)

Virtual Memory

Why Virtual Memory?

- A processor with a 32-bit address bus can address $2^{32} = 4$ GB of memory
- Physical DRAM may be only 1 GB (30-bit address bus)
- Programs need to run as if they have access to the full address space

Virtual memory uses main memory as a cache for secondary storage (hard disk).

$$\text{Virtual Address (VA)} \xrightarrow{\text{Translation}} \text{Physical Address (PA)}$$

- **Virtual addresses:** All addresses generated by the CPU (defined by ISA)
- **Physical addresses:** Actual locations in DRAM
- Data transfer between cache ↔ main memory: **block-wise**
- Data transfer between main memory ↔ disk: **page-wise**

Paging

Virtual memory is divided into fixed-size blocks called **pages** (always a power of 2).

$$\text{Virtual Address} = \underbrace{\text{Virtual Page Number (VPN)}}_{\text{upper bits}} \mid \underbrace{\text{Page Offset}}_{\text{lower bits}}$$

$$\text{Page offset bits} = \log_2(\text{Page size in bytes})$$

$$\text{Number of pages} = 2^{\text{VPN bits}}$$

Example: 16-bit VA, page size = 2 KiB:

$$\text{Page offset} = \log_2(2048) = 11 \text{ bits}$$

$$\text{VPN} = 16 - 11 = 5 \text{ bits} \Rightarrow 32 \text{ pages}$$

Typical page sizes today: 4 KiB to 64 KiB.

Page Table

The **Page Table** is the data structure that maps VPNs to physical page numbers (PPNs).

$$\text{Physical Address} = \text{PPN} \mid \text{Page Offset}$$

Key properties:

- Stored in **main memory**
- Each program has its **own page table**
- Indexed by VPN — no tags needed (every VPN has an entry)
- A **page table register** points to the start of the active page table
- Each entry contains:
 - **Valid bit (V):** V=1 → page is in memory; V=0 → page is on disk
 - **Physical Page Number** (if V=1)
 - **Disk address** (if V=0)
 - Access control bits (R/W/Execute/User-Supervisor)

Page Fault

A **page fault** occurs when V=0 — the requested page is not in physical memory.

Handling:

1. OS takes control via exception mechanism
2. OS finds the page on disk (using disk address from page table)
3. OS loads the page into a free physical frame in DRAM
4. OS updates the page table entry (V=1, PPN = new frame)
5. Instruction is restarted

Page fault cost: Millions of clock cycles (disk access) — this is why:

- Pages are **large** (4 KiB+) to amortize the high miss penalty
- **Fully associative** page placement is used (any page can go anywhere in physical memory) to minimize page fault rate
- Virtual memory uses **write-back** (write-through is too slow for disk)

Replacement policy: LRU (managed by OS in software — feasible because page faults are so rare that sophisticated algorithms are worth the cost).

Drawback of Simple Page Tables

Every memory access requires **two** main memory accesses:

1. Access page table to get physical address
2. Access actual data/instruction

This doubles the effective memory access time — unacceptable.

Solution: Translation Lookaside Buffer (TLB)

Translation Lookaside Buffer (TLB)

The TLB is a **small, fast, fully associative cache** inside the CPU that stores recently used VPN → PPN translations.

Why it works: Locality of reference applies to page table accesses too — once a page is accessed, it will likely be accessed again soon.

Why it's fast:

- Located inside the CPU (unlike page table which is in DRAM)
- Made of fast, expensive SRAM

TLB structure:

TLB TAG (VPN)	Valid	Dirty	Physical Page Number	Other bits
...	1	0	...	R/W/U/S

TLB Operation on every memory access:

1. Look up VPN in TLB
2. **TLB Hit:** Use PPN directly → form physical address → access cache/memory
3. **TLB Miss — two cases:**
 - *Merely a TLB miss:* Page is in memory but translation not in TLB → load translation from page table into TLB → retry
 - *True page fault:* Page is not in memory → invoke OS via exception

Important: TLB misses are far more frequent than true page faults, since TLB has far fewer entries than pages in memory.

Effective Access Time (with TLB, no page faults):

$$EAT = HR_{TLB} \times (TLB \text{ time} + MM \text{ time}) + MR_{TLB} \times (TLB \text{ time} + 2 \times MM \text{ time})$$

Where $MR_{TLB} = 1 - HR_{TLB}$.

Examples:

1. TLB search = 20 ns, MM access = 100 ns, EAT = 130 ns → find hit ratio:

$$130 = h(20 + 100) + (1 - h)(20 + 200) \Rightarrow h = 0.9$$

2. TLB search = 20 ns, hit ratio = 50%, MM = T ns, EAT = 170 ns:

$$170 = 0.5(20 + T) + 0.5(20 + 2T) \Rightarrow T = 100 \text{ ns}$$

TLB Miss Handling

When a TLB entry is replaced:

- Copy **reference bit** and **dirty bit** back to the page table entry
- These are the only fields that can change in a TLB entry
- Using write-back for this copying is efficient since TLB miss rate is low

Integrating VM, TLB, and Cache

Best case: TLB hit → Cache hit → data delivered to CPU in a few cycles

Worst case: TLB miss → Page fault → Cache miss → three separate stalls

Hierarchy of misses possible:

TLB	Page Table	Cache	Meaning
Hit	Not accessed	Hit	Normal fast path
Hit	Not accessed	Miss	Page in memory, data not in cache
Miss	Hit	Hit	Translation missing from TLB, page in memory, data in cache
Miss	Hit	Miss	Translation missing from TLB, page in memory, data not in cache
Miss	Miss (Page Fault)	Miss	Page not in memory

A cache hit can only occur after a TLB hit — you need the physical address before you can access a physically-indexed cache.

Protection: Virtual Memory and Virtual Machines

Protection via Virtual Memory

Problem: In multiprogramming (multiple programs sharing hardware), programs must be isolated — Process A should not read or overwrite Process B's memory.

Solution: Each process has its own page table. A process can only access physical pages mapped in its own page table.

Access Control per page table entry:

Bit	Meaning
R (Read)	Process can read this page
W (Write)	Process can write this page
U/S (User/Supervisor)	Page accessible in user mode or supervisor (kernel) mode only
Execute	Code can be executed from this page
Valid	Page is in memory

On translation: Access control bits are checked. If access is not permitted → **access protection fault** → exception handler invoked.

System Calls: A special instruction that transfers control to a dedicated location in *supervisor* code space, allowing a user process to request OS services safely.

Sharing: Two processes can share a physical page by having entries in their respective page tables point to the same physical page number.

Protection via Virtual Machines (VMs)

Motivation:

- Modern OS code bases have tens of millions of lines with thousands of bugs → security vulnerabilities
- Need a smaller, more auditable protection layer
- Multiple isolated environments on one machine (cloud/datacenter use case)

Virtual Machine Monitor (VMM) / Hypervisor:

- Software layer that sits between hardware and guest OS
- Much smaller than a full OS (~10,000 lines of isolation code)
- The underlying hardware is called the **host**; the VMs running on top are **guests**

Benefits of VMs:

1. **Security isolation** — each VM is isolated; a compromised guest can't affect others

2. **Software management** — run multiple OS versions simultaneously (legacy, stable, testing)
3. **Hardware consolidation** — multiple "servers" share one physical machine
4. **Live migration** — move a running VM between physical hosts for load balancing or maintenance

VMM Requirements:

- Guest software must behave as if on native hardware (except performance)
- Guest software cannot directly modify real hardware resource allocation
- VMM runs at a **higher privilege level** than guest VMs
- All privileged instructions must trap to VMM

VM and Virtual Memory:

- Each guest OS maintains its own page tables (virtual → real memory)
- VMM maintains **shadow page tables** mapping guest virtual → physical hardware memory directly
- Avoids paying an extra level of indirection on every memory access

Instruction-Level Parallelism (ILP)

What is ILP?

ILP is the ability to execute multiple instructions simultaneously by finding instructions that are **independent** of each other.

$$\text{Pipeline CPI} = \text{Ideal CPI} + \text{Structural stalls} + \text{Data hazard stalls} + \text{Control stalls}$$

Goal: Minimize stalls by exploiting ILP.

Loop-level parallelism is the most common source of ILP — different iterations of a loop are often fully independent.

```
for (i = 0; i <= 999; i++)  
    x[i] = x[i] + y[i];    // iteration i is independent of iteration j
```

Dependences

To exploit ILP, we must identify which instructions depend on each other.

- **Dependent instructions** must execute in order (may cause stalls)
- **Independent instructions** can execute simultaneously or be reordered

1. Data Dependences (True Dependences / RAW)

Instruction j is data dependent on instruction i if:

- i **produces** a result that j **consumes**, or
- There exists instruction k such that j depends on k and k depends on i (transitive)

```
i:  A = B + C    ← produces A
j:  D = E + A    ← consumes A    → j depends on i
```

Through registers: Straightforward to detect — register names are fixed.

Through memory: Harder — `100(R4)` and `20(R6)` might be the same address if R4 and R6 hold different base values.

A data dependence tells us three things:

1. The **possibility** of a hazard
2. The **order** in which results must be calculated
3. An **upper bound** on exploitable parallelism

Whether a dependence actually causes a stall depends on the pipeline organization, not just the program.

2. Name Dependences

Two instructions use the **same register or memory location** but there is **no actual data flow** between them. These can be resolved by **register renaming**.

Anti-dependence (WAR — Write After Read):

Instruction j *writes* a location that instruction i needs to *read* first.

```
i:  B = B + A    ← reads A
j:  A = C        ← writes A (must happen AFTER i reads A)
```

Output dependence (WAW — Write After Write):

Both i and j *write* the same location. The final written value must be from j .

```
i:  A = FMUL ...    ← writes A
j:  A = FADD ...    ← writes A (j's value must be the final one)
```

Since no value flows between instructions in name dependences, they can be eliminated by renaming the registers — allowing reordering.

Bernstein's Conditions for Parallel Execution

Two processes P1 and P2 (with inputs I_1, I_2 and outputs O_1, O_2) can execute in parallel **if and only if all three conditions hold**:

$I_2 \cap O_1 = \emptyset$ (no RAW: P2 doesn't read what P1 writes)

$I_1 \cap O_2 = \emptyset$ (no WAR: P1 doesn't read what P2 writes)

$O_1 \cap O_2 = \emptyset$ (no WAW: P1 and P2 don't both write the same location)

3. Control Dependences

An instruction is control dependent on a branch if its execution depends on the branch outcome.

Two constraints:

1. An instruction **control-dependent on a branch** cannot be moved *before* the branch (it might execute when it shouldn't)
2. An instruction **not control-dependent on a branch** cannot be moved *after* the branch (it might not execute when it should)

When can control dependences be violated safely?

Two properties must be preserved when reordering:

1. **Exception behavior** — reordering must not introduce new exceptions
2. **Data flow** — the correct data values must flow to consuming instructions

```
DADDU R1, R2, R3
BEQZ  R2, L1      ← branch
LW    R1, 0(R2)   ← control dependent on branch
```

Moving `LW` before `BEQZ` might cause a memory protection fault if $R2 = 0$. Even though there's no data dependence, control dependence prevents this move.

Safe to violate if: The moved instruction cannot raise an exception, and the destination register is **dead** (not used) after the label — so data flow is unaffected.

Data Hazards

A **hazard** exists when a dependence would cause incorrect results if the pipeline executes in overlap.

Three types of data hazards (for instructions i preceding j):

Hazard	Name	Description	Dependence Type
RAW	Read After Write	j reads before i writes	True data dependence

Hazard	Name	Description	Dependence Type
WAW	Write After Write	<i>j</i> writes before <i>i</i> writes, leaving <i>i</i> 's value	Output dependence
WAR	Write After Read	<i>j</i> writes before <i>i</i> reads, <i>i</i> gets wrong value	Anti-dependence

RAW (most common):

```
i: FADD F0, F1, F2    ← writes F0
j: FSUB F3, F0, F2    ← reads F0 (needs i to finish first)
```

WAW (occurs when multi-cycle operations allow out-of-order writes):

```
i: FMUL F0, F1, F2    ← takes 7 cycles, writes F0
j: FADD F0, F3, F4    ← takes 4 cycles, writes F0 (finishes before i!)
```

Result: F0 gets i's value instead of j's — wrong.

WAR (rare in static pipelines — reads are early, writes are late):

```
i: FSUB F3, F0, F2    ← reads F0
j: FADD F0, F1, F2    ← writes F0 (must not write before i reads)
```

Note: RAR (Read After Read) is never a hazard.

RISC-V Multi-Cycle Functional Units

Integer and FP operations have different latencies. The pipeline must handle variable-length execution stages.

Typical functional units:

1. **Main integer unit** — loads, stores, integer ALU, branches (1 cycle EX)
2. **FP/integer multiplier** (M1–M7, 7 stages)
3. **FP adder** (A1–A4, 4 stages)
4. **FP/integer divider** (variable latency)

Latency = number of cycles that must elapse between an instruction that produces a result and an instruction that uses it.

Producer → Consumer	Latency
fadd.d → fsub.d (FP add → FP add)	3 cycles
fadd.d → fsd (FP add → store)	2 cycles

Producer → Consumer	Latency
fld → fadd.d (load → FP add)	1 cycle
fld → fsd (load → store)	0 cycles

Basic Pipeline Scheduling

Goal: Rearrange instructions to separate dependent instructions by at least their latency — filling the gaps with independent instructions instead of stalls.

Example loop:

```
for (i = 999; i >= 0; i--)
    x[i] = x[i] + s;
```

Loop body:

```
fld    F0, 0(x1)    ← load x[i]
fadd.d F4, F0, F2    ← add scalar s (F2)
fsd    F4, 0(x1)    ← store result
addi   x1, x1, -8    ← decrement index
bne    x1, x2, Loop ← branch
```

Version	Stalls/iteration	Total cycles/element
Without scheduling	3 stalls	8 cycles
With scheduling	2 stalls	7 cycles

Scheduling moves independent instructions (like `addi`) between dependent pairs to fill stall slots.

Loop Unrolling

Idea: Replicate the loop body N times, process N elements per iteration → amortize loop overhead (branch + index update) over more work, and expose more independent instructions for scheduling.

Unrolled 4×:

```
fld    F0, 0(x1)
fld    F6, -8(x1)
fld    F10, -16(x1)
fld    F14, -24(x1)
fadd.d F4, F0, F2
fadd.d F8, F6, F2
```

```

fadd.d F12, F10, F2
fadd.d F16, F14, F2
fsd    F4, 0(x1)
fsd    F8, -8(x1)
fsd    F12, -16(x1)
fsd    F16, -24(x1)
addi   x1, x1, -32
bne    x1, x2, Loop

```

With scheduling (fill gaps between dependent FP instructions using loads from other iterations):

Version	Stalls/iteration	Cycles/element
No scheduling	3	8.0
Scheduled (no unrolling)	2	7.0
Scheduled + 4× unrolling	0	3.5

Comparison (1000 elements):

Version	Total cycles	Stall cycles	Overhead cycles
No scheduling	8000	3000	2000
With scheduling	7000	2000	2000
Scheduling + unrolling	3500	0	500

Quick Reference: Formulas

Formula	Expression
AMAT	$HT + MR \times MP$
AMAT (with way prediction)	$(HT_{DM} \times P_{correct} + HT_{SA} \times P_{wrong}) + MR_{SA} \times MP$
Pipeline CPI	Ideal CPI + Structural stalls + Data stalls + Control stalls
Effective Access Time (TLB)	$h(TLB + MM) + (1 - h)(TLB + 2 \times MM)$
Page offset bits	$\log_2(\text{Page size in bytes})$
Blocking improvement	$\frac{2N^3}{B} + N^2$ memory words (vs $2N^3 + N^2$ without blocking)
Bernstein condition 1	$I_2 \cap O_1 = \emptyset$

Formula	Expression
Bernstein condition 2	$I_1 \cap O_2 = \emptyset$
Bernstein condition 3	$O_1 \cap O_2 = \emptyset$